

Préparation au Concours National Commun — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

*Voyage d'un paquet : de l'adresse IP au cadenas du navigateur**Les protocoles TCP/IP et HTTP/HTTPS*Auteur : **Pr Agrégé El Hadiq Zouhair****cpgeacademy.org** — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit.

Sources. Le sujet s'appuie sur les documents normatifs de l'IETF et sur deux articles fondateurs : [RFC 791](#) (IP), [RFC 4632](#) (CIDR), [RFC 1071](#) et [RFC 1624](#) (somme de contrôle), [RFC 6298](#) (temporisateur de retransmission), [RFC 5681](#) (contrôle de congestion), [RFC 9110](#) et [RFC 9112](#) (HTTP/1.1), [RFC 9111](#) (cache), [RFC 8446](#) (TLS 1.3) ; V. Jacobson, *Congestion Avoidance and Control* (SIGCOMM 1988) et M. Mathis *et al.*, *The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm* (CCR 1997). **Aucune connaissance préalable en réseaux n'est requise** : tout ce qui est nécessaire est défini dans le sujet.

Pour aller plus loin. Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur [cpgeacademy.org](#).

Présentation générale

Lorsqu'un navigateur affiche une page, une longue chaîne de mécanismes s'est mise en marche : l'adresse de destination a été comparée à une table de routage, chaque paquet a été muni d'une somme de contrôle, un protocole de transport a garanti que rien n'a été perdu ni dupliqué, un texte a été analysé syntaxiquement, un cache a évité de retélécharger ce qui était déjà connu, et un échange cryptographique a établi un secret partagé.

Ce problème étudie **algorithmiquement** chacun de ces mécanismes. Il ne suppose *aucune* connaissance en réseaux : tous les objets manipulés sont définis dans le sujet, et chaque partie se réduit à un problème d'algorithmique, d'arithmétique, de graphes, de probabilités ou de bases de données.

Le sujet comporte **cinq parties largement indépendantes**, de difficulté croissante. Les fonctions écrites dans une question peuvent être librement réutilisées ensuite.

Notations et conventions

- Tous les entiers manipulés sont des entiers Python (précision arbitraire). Les opérateurs binaires & (et), | (ou), ^ (ou exclusif), << et >> (décalages) sont autorisés et supposés en $O(1)$ sur les entiers du sujet.
- Une **adresse IPv4** est un entier de 32 bits, noté aussi sous forme *décimale pointée* **a.b.c.d** où $a, b, c, d \in [0, 255]$, représentant l'entier $a \cdot 2^{24} + b \cdot 2^{16} + c \cdot 2^8 + d$.
- On note $\llbracket m, n \rrbracket$ l'ensemble des entiers k tels que $m \leq k \leq n$.

- En Python, seules les constructions du programme officiel sont autorisées : types de base, listes, tuples, dictionnaires, ensembles, boucles, fonctions, récursivité, classes simples, ainsi que les modules `math`, `random` et `heapq`. Les fonctions `sorted` et `sort` sont autorisées ; on admettra qu'elles trient m éléments en $O(m \log m)$.
- On admet que les opérations élémentaires sur un dictionnaire ou un ensemble (lecture, écriture, test d'appartenance) s'effectuent en $O(1)$ en moyenne.
- Sauf mention contraire, les complexités demandées sont des complexités *en temps* dans le pire des cas.

Le sujet comporte 5 parties et 60 questions. Les parties sont largement indépendantes ; à l'intérieur d'une partie, les questions s'enchaînent.

Partie I — Adressage IPv4 et agrégation CIDR

Références : [RFC 791](#) (Internet Protocol, 1981), [RFC 4632](#) (Classless Inter-domain Routing, 2006).

Un **préfixe** CIDR est un couple (p, n) , noté p/n , où $n \in \llbracket 0, 32 \rrbracket$ est la *longueur* du préfixe et p une adresse dont les $32 - n$ bits de poids faible sont nuls. Il désigne l'ensemble

$$\mathcal{A}(p/n) = \{a \in \llbracket 0, 2^{32} - 1 \rrbracket : a \text{ et } p \text{ ont les mêmes } n \text{ bits de poids fort}\}.$$

Par exemple $192.168.1.0/24$ désigne les 256 adresses de $192.168.1.0$ à $192.168.1.255$.

Question 1. Écrire deux fonctions `ip_vers_entier(s)` et `entier_vers_ip(x)` qui convertissent une chaîne en décimale pointée en l'entier correspondant, et réciproquement. Préciser leur complexité.

Question 2. Écrire une fonction `masque(n)` renvoyant, **sans boucle**, l'entier dont les n bits de poids fort valent 1 et les autres 0. Démontrer que $\text{masque}(n) = 2^{32} - 2^{32-n}$.

Question 3. Écrire `adresse_reseau(a, n)` et `diffusion(a, n)` renvoyant respectivement la plus petite et la plus grande adresse du préfixe de longueur n contenant a . Démontrer que $\text{diffusion}(a, n) = \text{adresse_reseau}(a, n) + 2^{32-n} - 1$.

Question 4. Dans un réseau IPv4, l'adresse de réseau et l'adresse de diffusion ne sont pas attribuables à une machine. En déduire le nombre d'adresses utilisables dans un préfixe de longueur $n \leq 30$, et démontrer la formule. Que se passe-t-il pour $n = 31$ et $n = 32$? Commenter.

Question 5. Écrire `dans_prefixe(a, p, n)` testant si $a \in \mathcal{A}(p/n)$, en utilisant **un seul** décalage et **un seul** ou exclusif. Démontrer l'équivalence

$$a \in \mathcal{A}(p/n) \iff (a \oplus p) \gg (32 - n) = 0 \quad (n \geq 1),$$

où $\oplus$ désigne le ou exclusif bit à bit.

Question 6. (Agrégation.) Soient p/n et q/n deux préfixes distincts de même longueur $n \geq 1$. Démontrer qu'il existe un préfixe $r/(n-1)$ tel que $\mathcal{A}(r/(n-1)) = \mathcal{A}(p/n) \cup \mathcal{A}(q/n)$ si et seulement si p et q ne diffèrent que par le bit de rang $32 - n$. Expliquer en une phrase l'intérêt de ce résultat pour la taille des tables de routage.

Question 7. (Famille laminaire.) Démontrer que deux préfixes CIDR quelconques p/n et q/m vérifient exactement l'une des trois propriétés suivantes : $\mathcal{A}(p/n) \subseteq \mathcal{A}(q/m)$, ou $\mathcal{A}(q/m) \subseteq \mathcal{A}(p/n)$, ou $\mathcal{A}(p/n) \cap \mathcal{A}(q/m) = \emptyset$. (Ce résultat est utilisé en partie III.)

Question 8. Applications numériques. Déterminer, en justifiant :

- (a) le masque de $/12$ en décimale pointée ;
- (b) l'adresse de réseau de $172.16.35.7/12$;
- (c) l'adresse de réseau, l'adresse de diffusion et le nombre d'adresses utilisables de $192.168.1.130/26$.

Partie II — La somme de contrôle Internet

Références : [RFC 1071](#) (Computing the Internet Checksum, 1988), [RFC 1624](#) (Computation of the Internet Checksum via Incremental Update, 1994).

Les protocoles IP, TCP et UDP protègent leurs en-têtes par une **somme de contrôle** de 16 bits reposant sur l'*addition en complément à un*. On note $N = 2^{16} = 65536$ et l'on travaille sur $\llbracket 0, N - 1 \rrbracket$.

Définition 1. L'**addition en complément à un** de $a, b \in \llbracket 0, N - 1 \rrbracket$, notée $a \oplus' b$, est obtenue en additionnant a et b puis en *réinjectant la retenue* dans les bits de poids faible (*end-around carry*) :

$$a \oplus' b = ((a + b) \bmod N) + \lfloor (a + b) / N \rfloor.$$

Le **complément** de x est $\bar{x} = x \oplus 0xFFFF$ (ou exclusif), soit $N - 1 - x$.

Question 9. Écrire une fonction `add16(a, b)` calculant $a \oplus' b$ en une ligne. Démontrer que le résultat appartient bien à $\llbracket 0, N - 1 \rrbracket$ (on prendra garde au cas où la réinjection provoque à son tour une retenue).

Question 10. Démontrer que pour tous $a, b \in \llbracket 0, N - 1 \rrbracket$ on a $a \oplus' b \equiv a + b \pmod{N - 1}$. En déduire que $\oplus'$ est **commutative** et **associative**, et que 0 en est un élément neutre. Que vaut $x \oplus' \bar{x}$? Commenter l'existence de *deux* représentations de zéro, `0x0000` et `0xFFFF`.

Question 11. Écrire `mots16(octets)`, qui regroupe une liste d'octets en mots de 16 bits (en complétant par un octet nul si le nombre d'octets est impair), puis `somme_un(mots)` et `checksum(octets) = somme_un(mots16(octets))`. Donner la complexité en fonction du nombre k d'octets.

Question 12. Le champ « somme de contrôle » est mis à zéro pour le calcul, puis remplacé par la valeur obtenue. Démontrer que le récepteur, en calculant `somme_un` sur le paquet *complet* (champ de contrôle inclus), doit trouver `0xFFFF`. Écrire la fonction `verifie(octets)` correspondante.

Question 13. (Indépendance de l'ordre des octets, rfc 1071 §2.B.) On note $\sigma(x)$ le mot obtenu en échangeant les deux octets de x . Démontrer que

$$\sigma(x_1) \oplus' \dots \oplus' \sigma(x_k) = \sigma(x_1 \oplus' \dots \oplus' x_k).$$

Quelle conséquence pratique cela a-t-il pour une machine dont l'ordre des octets natif diffère de celui du réseau ?

Question 14. Démontrer qu'une erreur de transmission portant sur **un seul bit** est toujours détectée.

Question 15. Montrer en revanche que la permutation de deux mots de 16 bits n'est *jamais* détectée, et exhiber une erreur portant sur *deux* bits qui passe inaperçue. Quelle propriété de $\oplus'$ est en cause ? Comparer brièvement avec un code polynomial de type CRC.

Question 16. (Mise à jour incrémentale.) Un routeur qui décrémente le champ « durée de vie » d'un paquet n'a pas besoin de recalculer toute la somme. On note HC l'ancienne somme de contrôle, m l'ancienne valeur du mot modifié, m' la nouvelle, et HC' la nouvelle somme.

- La [RFC 1141](#) proposait $HC' = HC \oplus' m \oplus' \overline{m'}$. Implémenter cette formule dans une fonction `maj_1141(HC, m, mp)`.
- La [RFC 1624](#) la corrige en $HC' = \overline{HC \oplus' \overline{m} \oplus' m'}$. Implémenter `maj_1624(HC, m, mp)`.
- On suppose que la somme en complément à un de tous les *autres* mots de l'en-tête vaut $C_{\text{autres}} = 0xCD7A$, que $m = 0x5555$ et $m' = 0x3285$. Calculer HC , puis HC' par recalcul complet, puis par chacune des deux formules. Conclure.
- Expliquer précisément *pourquoi* la formule de la [RFC 1141](#) échoue. On pourra invoquer la réponse à la question 10.

Question 17. Application numérique. On considère l'en-tête IPv4 suivant, donné en hexadécimal, dont le champ de contrôle (octets d'indices 10 et 11) a été mis à zéro :

45 00 00 73 00 00 40 00 40 11 00 00 c0 a8 00 01 c0 a8 00 c7

Calculer la somme de contrôle à y inscrire. Vérifier ensuite que le paquet complet satisfait bien le test de la question 12.

Partie III — Routage : plus long préfixe et plus courts chemins

Références : [RFC 4632](#) (CIDR), [RFC 2328](#) (OSPF, algorithme à état de liens), [RFC 2453](#) (RIP, algorithme à vecteur de distance).

III.1 — Recherche du plus long préfixe correspondant

Une **table de routage** est une liste de triplets (p, n, saut) . Pour router une adresse a , le routeur retient le préfixe *le plus long* parmi ceux qui contiennent a , et transmet le paquet au saut associé. Le préfixe $0.0.0.0/0$, présent dans toute table, joue le rôle de route par défaut.

Question 18. Écrire `plus_long_prefixe(table, a)` par parcours exhaustif de la table. Donner sa complexité en fonction du nombre k d'entrées.

Question 19. Démontrer, à l'aide de la question 7, que le préfixe le plus long contenant a est **unique** dès lors que la table ne contient pas deux entrées de même préfixe.

Question 20. On représente maintenant la table par un **arbre binaire de préfixes** (ou *trie*) : un nœud à la profondeur d représente une suite de d bits ; son fils gauche (resp. droit) représente cette suite suivie du bit 0 (resp. 1). Un nœud porte un saut si et seulement si la suite de bits correspondante est un préfixe de la table. Écrire une méthode `insérer(p, n, saut)`. Donner la complexité en temps d'une insertion et la complexité en espace de la structure pour k entrées.

Question 21. Écrire la méthode `chercher(a)` qui descend dans l'arbre en suivant les bits de a et renvoie le saut du *dernier* nœud rencontré qui en porte un. Énoncer l'invariant de la boucle et démontrer la correction.

Question 22. Donner la complexité de `chercher`. Comparer avec la question 18 et commenter la pertinence de cette structure pour un routeur de cœur de réseau, dont la table comporte aujourd'hui environ 10^6 entrées.

III.2 — Plus courts chemins dans le graphe du réseau

On modélise le réseau par un graphe non orienté pondéré $G = (S, A)$: les sommets sont les routeurs, numérotés de 0 à $V - 1$, et le poids d'une arête est le *coût* du lien (entier strictement positif). On note $E = |A|$.

Question 23. Écrire `dijkstra(V, adj, s)` calculant les distances de s à tous les sommets, où `adj[u]` est la liste des couples (v, w) . On utilisera le module `heapq`. Donner la complexité.

Question 24. Démontrer la correction de l'algorithme de Dijkstra en énonçant l'invariant vérifié à chaque extraction du tas. Où l'hypothèse « poids positifs » est-elle utilisée ?

Question 25. Écrire `bellman_ford(V, aretes, s)`, où `aretes` est la liste des arcs (u, v, w) . La fonction renverra `None` si le graphe contient un circuit de poids strictement négatif. Donner la complexité.

Question 26. Démontrer, par récurrence sur i , qu'après i tours de la boucle externe, `d[v]` est majoré par le poids du plus court chemin de s à v utilisant au plus i arêtes. En déduire la correction et justifier le nombre $V - 1$ de tours.

Question 27. (Comptage à l'infini.) Le protocole RIP est un algorithme à vecteur de distance : chaque routeur ne connaît que les distances annoncées par ses voisins. Sur le graphe réduit à trois routeurs $A - B - C$ en ligne (arêtes de poids 1), décrire ce qui se produit lorsque le lien $B - C$ tombe, et expliquer pourquoi les distances annoncées croissent indéfiniment. Quelle contre-mesure élémentaire RIP adopte-t-il ? Commenter son coût.

Question 28. Application numérique. On considère le graphe non orienté à 6 sommets d'arêtes

$(0, 1, 7) (0, 2, 9) (0, 5, 14) (1, 2, 10) (1, 3, 15) (2, 3, 11) (2, 5, 2) (3, 4, 6) (4, 5, 9)$.

Calculer, en détaillant les étapes de Dijkstra, les distances depuis le sommet 0, ainsi qu'un plus court chemin de 0 à 4.

Partie IV — TCP : fiabilité, temporisateur et congestion

Références : RFC 9293 (TCP), RFC 6298 (Computing TCP's Retransmission Timer, 2011), RFC 5681 (TCP Congestion Control, 2009), V. Jacobson, Congestion Avoidance and Control, SIGCOMM 1988 ; M. Mathis, J. Semke, J. Mahdavi, T. Ott, The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm, ACM CCR 27(3), 1997.

IV.1 — Numéros de séquence circulaires

TCP numérote les octets par un compteur de 32 bits qui *boucle* : on travaille dans $\mathbb{Z}/M\mathbb{Z}$ avec $M = 2^{32}$. On veut décider si un numéro a « précède » un numéro b .

Question 29. On pose $\text{avant}(a, b) = [(b - a) \bmod M < 2^{31} \text{ et } a \neq b]$. Écrire la fonction. Démontrer que si tous les numéros en circulation à un instant donné appartiennent à une fenêtre de largeur W , alors avant rend le verdict correct dès que $W < 2^{31}$, et exhiber une ambiguïté lorsque $W = 2^{31}$.

IV.2 — Réassemblage des segments

Un segment reçu couvre l'intervalle d'octets $[d, f[$. Les segments arrivent dans le désordre et peuvent se chevaucher. Le récepteur doit maintenir la liste des zones effectivement reçues.

Question 30. Écrire $\text{fusionner}(\text{segments})$ qui, à partir d'une liste de couples (d, f) , renvoie la liste des intervalles maximaux couverts, triée par début croissant. Donner la complexité en fonction du nombre k de segments.

Question 31. Démontrer la correction de fusionner : la réunion des intervalles renvoyés est égale à celle des intervalles fournis, et les intervalles renvoyés sont deux à deux disjoints et *non adjacents*. On énoncera un invariant de boucle.

IV.3 — Estimation du temporisateur de retransmission

Le RFC 6298 prescrit l'algorithme suivant, dû à Jacobson et Karels. À la première mesure de temps d'aller-retour R :

$$\text{SRTT} \leftarrow R, \quad \text{RTTVAR} \leftarrow R/2,$$

puis, à chaque mesure ultérieure R' , dans cet ordre :

$$\text{RTTVAR} \leftarrow (1 - \beta) \text{RTTVAR} + \beta |\text{SRTT} - R'|, \quad \text{SRTT} \leftarrow (1 - \alpha) \text{SRTT} + \alpha R',$$

avec $\alpha = 1/8$ et $\beta = 1/4$; enfin $\text{RTO} \leftarrow \max(1, \text{SRTT} + 4 \text{RTTVAR})$ (en secondes).

Question 32. Écrire deux fonctions $\text{rto_initial}(R)$ et $\text{rto_suivant}(\text{SRTT}, \text{RTTVAR}, R)$ implémentant ces règles. Pourquoi l'ordre des deux affectations est-il important ?

Question 33. On note R_0 la première mesure et $R_1, \dots, R_n$ les suivantes, et S_n la valeur de SRTT après la n -ième mise à jour. Démontrer par récurrence que

$$S_n = (1 - \alpha)^n R_0 + \alpha \sum_{i=1}^n (1 - \alpha)^{n-i} R_i.$$

Vérifier que la somme des coefficients vaut exactement 1.

Question 34. On appelle *mémoire effective* de l'estimateur le nombre $\mu = \sum_{j \geq 0} (j+1) \alpha (1 - \alpha)^j$ d'échantillons « équivalents » qu'il retient. Calculer μ en fonction de α , puis sa valeur pour $\alpha = 1/8$. Déterminer également le plus petit entier j tel que le poids $\alpha (1 - \alpha)^j$ soit inférieur au dixième du poids le plus récent. Interpréter.

Question 35. (Algorithme de Karn et repli exponentiel.) Lorsqu'un segment est retransmis, on ne peut pas savoir si l'accusé de réception répond à la première ou à la seconde émission. Expliquer en quoi mesurer R dans ce cas biaiserait l'estimateur, et dans quel sens. Le [RFC 6298](#) impose de plus $RTO \leftarrow 2RTO$ à chaque expiration. Calculer le temps d'attente cumulé après n expirations successives à partir d'un RTO initial T_0 , et donner sa valeur pour $T_0 = 1$ s et $n = 5$.

Question 36. On suppose que chaque transmission d'un segment échoue indépendamment avec probabilité $p \in]0, 1[$. Soit N le nombre total de transmissions nécessaires. Reconnaître la loi de N , donner $\mathbb{P}(N = k)$, $\mathbb{P}(N > k)$ et $\mathbb{E}[N]$. Démontrer que la transmission aboutit **presque sûrement**. Application numérique pour $p = 0,1$.

IV.4 — Contrôle de congestion : le débit d'une connexion TCP

TCP règle son débit par une *fenêtre de congestion* W (en segments). En régime d'évitement de congestion ([RFC 5681](#)), W augmente de 1 à chaque aller-retour ; à chaque perte, W est divisée par 2. C'est le schéma dit **AIMD** (*additive increase, multiplicative decrease*). On suppose le régime périodique : W croît de $W_{\max}/2$ à $W_{\max}$, une perte survient, et le cycle recommence. On note RTT la durée d'un aller-retour, supposée constante, et $W_{\max}$ pair.

Question 37. Démontrer que le nombre de segments émis pendant un cycle vaut exactement

$$N_{\text{cycle}} = \frac{3W_{\max}^2}{8} - \frac{W_{\max}}{4},$$

et que la fenêtre moyenne sur un cycle vaut $\bar{W} = \frac{3W_{\max} - 2}{4}$.

Question 38. On suppose qu'exactly une perte se produit par cycle, et l'on note p la probabilité de perte d'un segment, de sorte que $p = 1/N_{\text{cycle}}$. En négligeant les termes d'ordre 1 devant les termes d'ordre 2, démontrer la **formule de Mathis** :

$$W_{\max} \approx \sqrt{\frac{8}{3p}}, \quad \text{d'où} \quad \text{débit} \approx \frac{MSS}{RTT} \sqrt{\frac{3}{2p}} \approx \frac{1,22 MSS}{RTT\sqrt{p}} \quad (\text{octets/s}),$$

où MSS est la taille d'un segment en octets.

Question 39. Application numérique : $MSS = 1460$ octets, $RTT = 100$ ms. Calculer le débit prévu et la fenêtre $W_{\max}$ pour $p = 10^{-2}$ puis pour $p = 10^{-4}$.

Question 40. Commenter la formule de Mathis. En particulier : (a) quel est l'effet d'une division par 4 du taux de perte ? (b) deux connexions partageant le même lien mais de RTT différents obtiennent-elles la même part du débit ? Quelle conséquence pour l'équité ?

Partie V — HTTP, cache et HTTPS

Références : [RFC 9110](#) et [RFC 9112](#) (HTTP/1.1, 2022), [RFC 9111](#) (cache HTTP), [RFC 8446](#) (TLS 1.3, 2018) ; W. Diffie et M. Hellman, New Directions in Cryptography, *IEEE IT-22*(6), 1976.

V.1 — Analyse d'une requête HTTP

Une requête HTTP/1.1 est un texte de la forme

```
GET /index.html?p=1 HTTP/1.1\r\n
Host: cpgeacademy.org\r\n
Content-Length: 5\r\n
\r\n
HELLO
```


La première ligne (*ligne de requête*) contient trois champs séparés par une espace. Suivent des *en-têtes* de la forme `Nom: valeur`, un par ligne. Une ligne vide, c'est-à-dire la séquence `CRLF CRLF` (soit `"\r\n\r\n"`), sépare l'en-tête du corps.

Question 41. Écrire `analyser_requete(texte)` renvoyant le quintuplet

(méthode, cible, version, dictionnaire des en-têtes, corps),

les noms d'en-têtes étant mis en minuscules. Donner la complexité.

Question 42. Un serveur lit le flux *caractère par caractère* et doit détecter la fin de l'en-tête sans jamais revenir en arrière. Construire un **automate fini déterministe** à quatre états reconnaissant la première occurrence de `CRLFCRLF`. Donner sa table de transition, puis écrire la fonction `fin_entete(flux)` renvoyant l'indice suivant la fin de l'en-tête, ou `-1`.

Question 43. Démontrer la correction de cet automate en énonçant l'invariant : *à chaque instant, l'état courant est la longueur du plus long suffixe du texte déjà lu qui soit un préfixe de CRLFCRLF*. En déduire que l'analyse est linéaire en la longueur du flux et ne nécessite aucun retour en arrière. Vérifier l'invariant sur l'entrée `"\r\n\r\n\r\n\r\n"`.

Question 44. Lorsque la taille du corps n'est pas connue à l'avance, HTTP/1.1 utilise le codage *par morceaux* : le corps est une suite de blocs, chacun précédé de sa taille écrite en hexadécimal et suivi de `CRLF` ; un bloc de taille nulle termine le corps. Écrire `decoder_chunked(s)`. Exhiber un variant de boucle et en déduire la terminaison. Donner la complexité.

V.2 — Le cache

Question 45. Un cache de capacité C objets applique la politique **LRU** (*least recently used*) : en cas de saturation, l'objet dont le dernier accès est le plus ancien est évincé. Proposer une structure de données permettant de traiter un accès en $O(1)$ *en moyenne*, et écrire la classe correspondante avec une méthode `accéder(cle)` renvoyant `True` en cas de succès. Justifier la complexité et énoncer l'invariant reliant la structure à l'ordre des derniers accès.

Question 46. Dérouler la politique LRU de capacité 3 sur la suite d'accès

$a, b, c, a, d, b, a, c, d, a$

en indiquant à chaque étape le contenu du cache, et donner le nombre de succès.

V.3 — Choix optimal du contenu d'un cache (programmation dynamique)

Un serveur dispose de n objets ; l'objet i occupe t_i octets et est demandé f_i fois par heure. On veut choisir un sous-ensemble S d'objets à placer en mémoire, de taille totale au plus C , maximisant le nombre de requêtes servies depuis le cache :

$$\mu(t, f, C) = \max \left\{ \sum_{i \in S} f_i : S \subseteq \{0, \dots, n-1\}, \sum_{i \in S} t_i \leq C \right\}.$$

Question 47. On pose, pour $0 \leq i \leq n$ et $0 \leq j \leq C$, $m_i(j)$ le nombre maximal de requêtes servies par un sous-ensemble de $\{0, \dots, i-1\}$ de taille totale au plus j . Donner m_0 , puis établir la relation de récurrence exprimant m_{i+1} en fonction de m_i .

Question 48. Écrire `cache_optimal(t, f, C)` implémentant cette récurrence avec un **unique** tableau de taille $C+1$ mis à jour en place. Justifier précisément le **sens de parcours** de j , et montrer sur un exemple explicite ce que renverrait l'algorithme si l'on parcourait dans l'autre sens.

Question 49. Démontrer la correction en énonçant et en prouvant, par récurrence sur le nombre i d'objets traités, l'**invariant** vérifié par le tableau à la fin de la i -ième itération de la boucle externe. Justifier la terminaison.

Question 50. Donner la complexité en temps et en espace. Le problème de décision associé est SAC-À-DOS, connu NP-complet ; montrer qu'il appartient à NP en explicitant un certificat et un vérificateur. Expliquer pourquoi l'algorithme précédent ne contredit pas cette NP-complétude, et ce qui se produit si les tailles sont exprimées en octets et $C = 10$ Go.

Question 51. Application numérique. On prend $t = [3, 4, 5, 9, 4]$, $f = [60, 50, 70, 90, 30]$ (tailles en Mo) et $C = 10$. Calculer μ et donner un sous-ensemble optimal. Détailler le contenu du tableau à la fin de chaque itération pour la version réduite $t = [3, 4, 5]$, $f = [60, 50, 70]$, $C = 8$.

V.4 — Analyse des journaux du serveur (SQL)

Le serveur enregistre ses requêtes dans une base relationnelle de schéma :

```
journal(id, ip, horodatage, methode, chemin, statut, octets, duree_ms)
client(ip, pays, organisation)
```

où `horodatage` est une chaîne au format 'AAAA-MM-JJ HH:MM:SS' et `statut` le code de réponse HTTP (2xx : succès, 3xx : redirection ou contenu inchangé, 4xx : erreur du client, 5xx : erreur du serveur). On rappelle que `substr(s, i, k)` extrait k caractères de s à partir de la position i (comptée à partir de 1).

Question 52. Écrire une requête SQL donnant, pour chaque méthode, le nombre de requêtes et le volume total d'octets transmis, par nombre de requêtes décroissant.

Question 53. Écrire une requête donnant les trois chemins les plus demandés, avec leur nombre de requêtes.

Question 54. Écrire une requête donnant, pour chaque heure de la journée, le nombre total de requêtes, le nombre de réponses 5xx et le taux d'erreur en pourcentage arrondi au dixième.

Question 55. Écrire une requête donnant les adresses IP ayant provoqué au moins trois réponses 4xx, avec le nombre correspondant.

Question 56. Écrire une requête donnant, par pays, le nombre de requêtes et le volume d'octets. Écrire enfin une requête donnant la liste des chemins qui n'ont *jamaïs* été servis avec le statut 200.

V.5 — HTTPS : l'établissement du secret partagé

TLS 1.3 (RFC 8446) établit un secret partagé par un échange de Diffie–Hellman. On se donne un nombre premier p et un entier $g \in \llbracket 2, p-2 \rrbracket$, tous deux publics.

Question 57. Écrire une fonction `puissance_mod(a, e, n)` calculant $a^e \bmod n$ par exponentiation rapide, sans utiliser `pow(a, e, n)`. Exhiber un variant de boucle et en déduire la terminaison. Démontrer la correction en énonçant l'invariant $r \cdot a^e \equiv a_0^{e_0} \pmod{n}$. Démontrer que le nombre de multiplications modulaires est au plus $2\lfloor \log_2 e \rfloor + 2$.

Question 58. Le client tire x au hasard et envoie $A = g^x \bmod p$; le serveur tire y et envoie $B = g^y \bmod p$. Chacun calcule alors une clé. Écrire les deux calculs et démontrer qu'ils donnent le même résultat. Application numérique : $p = 23$, $g = 5$, $x = 6$, $y = 15$; calculer A , B et la clé partagée.

Question 59. Un attaquant qui observe le réseau connaît p , g , A et B . Le retrouver x est le problème du **logarithme discret**. Décrire un algorithme naïf et donner sa complexité. Décrire ensuite le principe de l'algorithme « pas de bébé, pas de géant » en $O(\sqrt{p})$ opérations et $O(\sqrt{p})$ en espace. Pour p de 2048 bits, comparer les deux coûts au nombre d'opérations réalisables en un an par une machine effectuant 10^9 opérations par seconde. Conclure.

Question 60. Question de synthèse (à traiter en quelques lignes). TLS 1.3 impose que x et y soient *éphémères*, c'est-à-dire tirés à nouveau pour chaque connexion et effacés ensuite, et a supprimé la possibilité d'un échange de clés reposant sur le chiffrement par la clé publique

du serveur. Expliquer ce que l'on gagne ainsi si la clé privée du serveur est compromise *après* l'interception et l'enregistrement d'une session. En quoi la partie II (intégrité) et cette partie V (confidentialité) répondent-elles à des menaces de nature différente ?

FIN DE L'ÉPREUVE